UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
Departamento de Computação – DC
Bacharelado em Ciência da Computação
Disciplina: Sistemas Distribuídos
Prof.: Paulo Henrique Lopes Silva

Prática Offline 1

1. Nomeação em Sistemas Distribuídos: Conceitos Fundamentais e Aplicações.

- **Conceito geral de nomeação.**
    - Em qualquer sistema distribuído, há recursos (processos, arquivos, serviços, dispositivos, nós, objetos remotos) que precisam ser identificados e localizados.
    - O mecanismo que permite isso é o **sistema de nomeação** *(naming system)*.
    - **Nomeação** é o processo de atribuir nomes simbólicos a recursos distribuídos e permitir que esses nomes sejam traduzidos em endereços ou identificadores reais.
    - Em outras palavras:
        - **Nome**: representação simbólica compreensível (ex.: "servidor1", "db.local").
            - Um rótulo simbólico usado para referenciar algo.
        - **Endereço**: representação física (ex.: "192.168.0.10", "10.0.0.15:8080");
            - Localização física ou lógica do recurso.
        - **Identificador**: código único interno (ex.: UUID, hash, *handle*).
            - Um valor único que identifica um recurso, independente da localização.
    - Esses três conceitos estão interligados, mas não são a mesma coisa.
        - Um nome pode mudar de endereço ao longo do tempo, e um mesmo endereço pode conter diferentes nomes (ex.: servidor virtualizado).

- ***Binding* (ligação).**
    - O *binding* é a associação entre nome, identificador e endereço.
    - Há três tipos principais:
        - **Estaticamente ligado:** Feito uma vez e nunca muda.
            - Ex.: "servidor1": 192.168.0.10" fixo no código.
        - **Ligação tardia *(late binding):*** Feita em tempo de execução, mas raramente alterada.
            - Ex.: Cliente consulta DNS a cada conexão.
        - **Ligação dinâmica *(dynamic binding)*:** Atualizada automaticamente conforme o recurso muda.
            - Serviço em nuvem que muda de IP e se registra de novo.
        - Em sistemas modernos (microsserviços, nuvem, IoT), o ***binding* dinâmico** é o mais usado, pois os serviços são elásticos e efêmeros.

- **Objetivos do sistema de nomeação.**
    - **Abstração da localização:**
        - O cliente não precisa saber onde o recurso está, apenas quem ele é.
    - **Transparência de mobilidade:**
        - Se o recurso se mover (mudar de IP ou servidor), o nome continua o mesmo.
    - **Escalabilidade:**
        - O sistema deve suportar bilhões de nomes e resolvê-los eficientemente (ex.: DNS global).
    - **Tolerância a falhas:**
        - O sistema deve funcionar mesmo que partes da infraestrutura falhem.
    - **Desempenho:**
        - Resolução rápida, caches e replicação de nomes para evitar gargalos.

- **Tipos de sistemas de nomeação.**
    - **Nomeação plana *(Flat naming)*.**
        - Cada recurso tem um identificador único (ex.: hash, UUID).
        - Sem estrutura hierárquica, busca é feita por difusão ou DHT.
        - Muito comum em sistemas P2P (Chord, Pastry, Kademlia).
        - Exemplo: Cada nó na rede tem um identificador SHA-1 e armazena pares chave → valor.
    - **Nomeação hierárquica.**
        - Nomes organizados em níveis (semelhante a diretórios).
        - Permite delegação de autoridade.
        - Base do DNS *(Domain Name System)*.
        - Exemplo:
        - www.instituto.ufersa.br
            - br → nível raiz.
            - ufersa → subdomínio.
            - instituto → unidade.
            - www → servidor web (Tradicionalmente aponta para o serviço HTTP/HTTPS).
    - **Nomeação baseada em contexto (ou contexto de nomes).**
        - Um mesmo nome pode significar coisas diferentes em contextos diferentes.
        - Muito usada em sistemas de objetos distribuídos (ex.: *RMI Registry*, *CORBA Naming Service*).
    - **Nomeação de conteúdo *(content-based)*.**
        - O nome é derivado do conteúdo (hash de um arquivo ou bloco de dados).
        - Usada em sistemas imutáveis e redes de conteúdo (CDNs, IPFS).
        - Exemplo:
        - O arquivo é identificado por SHA-256("conteúdo").

Questões práticas.

1. Mini-DNS (resolução de nomes): Implementar um servidor simples que mantém um mapa.

```
Java
"servidor1" → "192.168.0.10"
"servidor2" → "192.168.0.20"
"servidor3" → "192.168.0.30"
"servidor4" → "192.168.0.40"
"servidor5" → "192.168.0.50"
"servidor6" → "192.168.0.60"
"servidor7" → "192.168.0.70"
"servidor8" → "192.168.0.80"
"servidor9" → "192.168.0.90"
"servidor10" → "192.168.0.100"
```

- O cliente pergunta pelo nome e o servidor devolve o endereço.
    - Considere a existência de três clientes requisitantes.
- Depois, simular ***binding dinâmico***: um cliente do tipo registrador atualiza os endereços associados a "servidor1", "servidor4" e "servidor9".
    - O servidor envia a atualização aos clientes requisitantes. Mostrar que os clientes recebem o novo valor.

2. Descoberta de serviços *(service discovery)*: Criar servidores que oferecem um serviço simples (ex.: “Calcular soma”, “Calcular subtração”, “Calcular multiplicação” e “Calcular divisão”).

- Dois servidores possuem o serviço de calculadora básica, com as operações listadas (cada qual com seu nome e endereço simulados).
- Um servidor de diretório registra todos os servidores disponíveis.
- Considere três clientes:
    - Um cliente consulta o diretório para descobrir qual servidor está ativo.
    - Faça testes de descoberta com, pelo menos, dois serviços.
- Incluir ***load balancing***:
    - Um cliente pega um endereço entre os registrados, de acordo com uma das técnicas.
        - *Round robin*.
        - *Random*.

3. Nomeação em P2P (*overlay* simplificado): Desenvolva uma aplicação cliente/servidor *multithread*, com TCP ou UDP, para simular a comunicação de processos (nós) em uma topologia em anel. Considere seis processos chamados de P0, P1, P2, P3, P4 e P5. Considere o seguinte esquema:

- Cada nó tem um identificador numérico.
- O nó sabe quem é seu sucessor e seu antecessor.
- Para encontrar um recurso (ex.: arquivo “X”), um nó repassa a mensagem ao próximo até o responsável ser encontrado.
    - Os arquivos podem ser simulados com strings: arquivo1, arquivo2, arquivo23, arquivo47, etc.
    - A rede armazena 60 arquivos, sendo:
        - P0 com os arquivos de 1 a 10.
        - P1 com os arquivos de 11 a 20.
        - P2 com os arquivos de 21 a 30.
        - P3 com os arquivos de 31 a 40.
        - P4 com os arquivos de 41 a 50.
        - P5 com os arquivos de 51 a 60.
- Esse exercício mostra nomeação distribuída sem servidor central.
- Observações:
    - As mensagens de busca por um “arquivo” podem ser enviadas por qualquer processo.
        - Você pode definir o sentido das mensagens (horário ou anti-horário).
- Sobre as mensagens:
    - As mensagens podem ser de tipos primitivos ou compostos.
    - Para entradas erradas, o programa deve imprimir *“Erro na entrada de dados. Tente outra vez!”*.
    - Cada processo deve armazenar e imprimir na tela um log de mensagens recebidas e enviadas.

Observações
● Data e hora de entrega da tarefa: 30/10/2025 até às 23h59. Portanto, certifiquem-se do arquivo que vão enviar. ● Crie uma pasta chamada “seu-nome-pratica-off1”, comprima e envie pelo SIGAA (única forma de envio). ● Avaliação: o projeto vale 60% da nota da 2ª unidade. ● Os projetos são individuais. ● Os projetos devem utilizar as tecnologias vistas na disciplina (*Threads*, *Sockets* TCP e UDP). ● Casos de punição com a nota zero, para os envolvidos.   ○ Respostas ou códigos iguais.   ○ Respostas ou códigos sem a devida explicação. ● Bom trabalho!

Roteiro para Gravação da Apresentação

1. Executar o programa resultante de cada enunciado, questão ou projeto.
2. Para cada programa, execute uma operação por vez.
    - Durante a testagem de cada funcionalidade, comente sobre os possíveis erros e exceções que podem ocorrer mediante entradas incorretas. Comente também sobre quais erros e exceções seu programa captura.
    - Cite as ferramentas e tecnologias utilizadas.
    - Seu programa implementa todas as funcionalidades requeridas? Em caso negativo, justifique.
3. Após a demonstração de funcionamento, é hora de abrir o código.
    - Explique a organização das respostas.
    - Houve alguma dificuldade ou detalhe interessante na implementação de algum dos requisitos mencionados?
    - Quais as principais dificuldades e lições aprendidas com o desenvolvimento do projeto?
    - Responda as perguntas detalhando o código que você implementou.
4. Detalhamento das instruções:
    - O vídeo deve ter no máximo 12 minutos.
    - A gravação será o instrumento avaliativo.
    - Pontos de avaliação:
        - Sequência lógica e domínio do conteúdo (introdução, divisão organizada da apresentação, concatenação de ideias e conclusão).
        - Precisão na comunicação (colocação e entonação da voz, ritmo, dicção e linguagem).
        - Capacidade de argumentação.
        - Domínio e uso de material (organização no uso de recursos de apresentação).
6. Sugestões de software para gravação:
    - Google Meet, Zoom, SimpleScreenRecorder, OBS, etc.